Завдання 1

```
--створюємо тимчасову таблицю temp_logs
CREATE TEMP TABLE temp_logs
AS
SELECT user_id,
STRUCT(event_name AS event, screen) AS details
FROM `homework_bigquery.event_logs`;

--виводимо окремо вкладені поля details.event та details.screen
SELECT
  user_id,
  details.event,
  details.screen
FROM temp_logs;
```

Query results

Job information | Results | Visualization | JSON | Execution details | Executi

Row	user_id	event	screen
1	user_003	add_to_cart	cart
2	user_002	add_to_cart	cart
3	user_001	add_to_cart	cart
4	user_004	add_to_cart	cart
5	user_001	purchase	checkout
6	user_004	purchase	checkout
7	user_002	purchase	checkout
8	user_002	open_app	home
9	user_004	open_app	home

Завдання 2

```
--рахуємо кількість унікальних користувачів,
--які здійснили подію purchase за останні 3 дні включно з сьогодні з фільтром по
_PARTITIONDATE
SELECT COUNT(DISTINCT user_id) AS total_users
FROM `homework_bigquery.event_logs`
WHERE
_PARTITIONDATE BETWEEN CURRENT_DATE()-2 AND CURRENT_DATE()
AND event_name='purchase';
```

Query results

Job information | Results

Row	total_users
1	3

Завдання 3

```
--побудова масиву дій кожного користувача у межах дня
SELECT user_id,
DATE(event_time) AS event_date,
ARRAY_AGG(event_name) AS events_per_day
FROM `homework_bigquery.event_logs`
GROUP BY user_id, event_date;
```

Query results

Job information | Results | Visualization | JSON | Execution details

Row	user_id	event_date	events_per_day
1	user_001	2024-06-01	open_app
			search
			view_product
			add_to_cart
			purchase
2	user_002	2024-06-01	open_app
			view_product
			add_to_cart
			purchase

Завдання 4

```
--створення CTE зі STRUCT на базі наявних полів
WITH user_actions AS
(
SELECT user_id,
STRUCT(event_name, screen) AS details
FROM `homework_bigquery.event_logs`
)

--виводимо user_id as uid, screen as location
SELECT user_id as uid,
details.screen AS location
FROM user_actions;
```

Query results

Job information | Results | Visualization | JSON | E

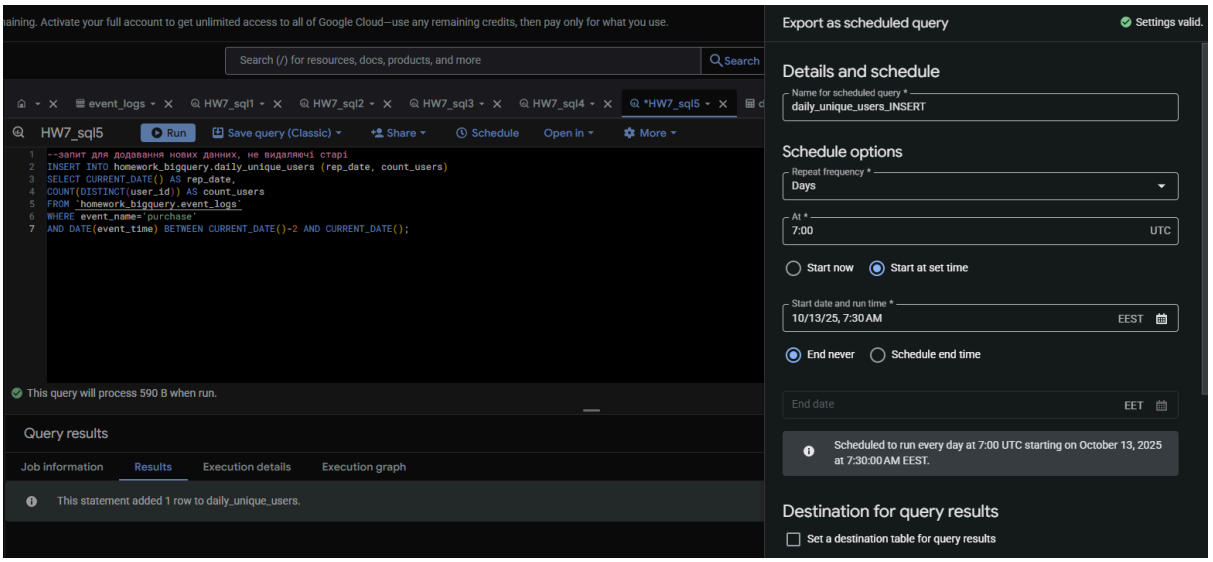
Row	uid	location
1	user_001	home
2	user_001	search
3	user_001	product_card
4	user_001	cart
5	user_001	checkout
6	user_002	home
7	user_002	product_card
8	user_002	cart
9	user_002	checkout

Завдання 5

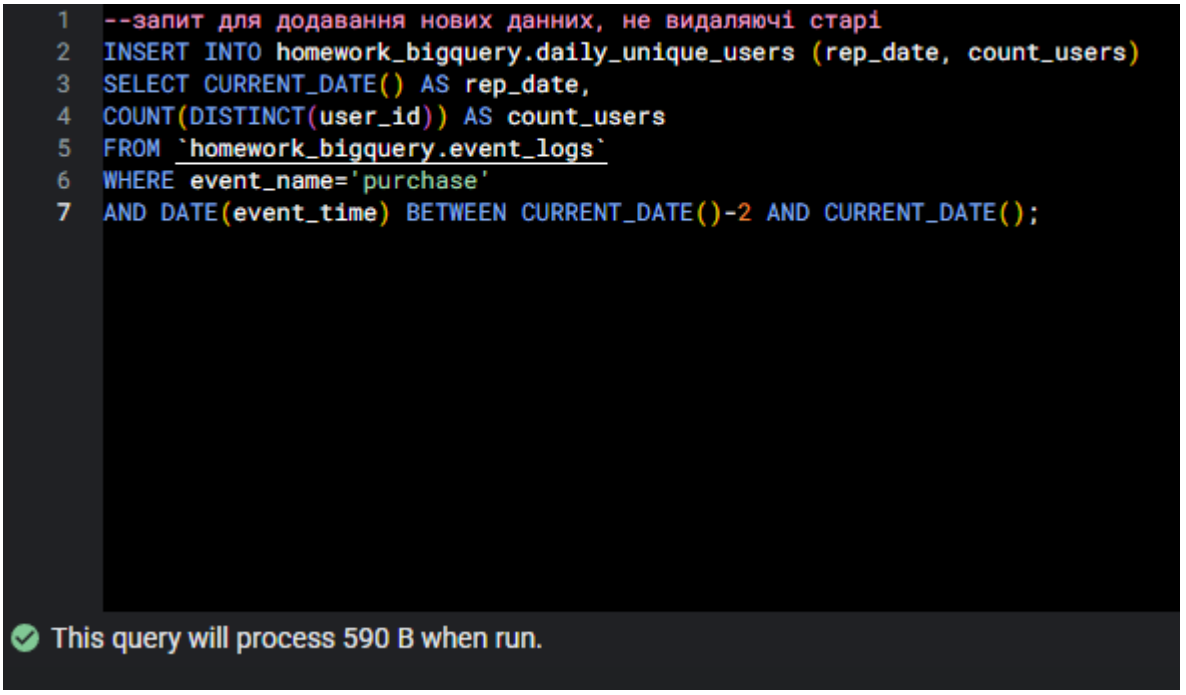
```
--створюємо таблицю для зберігання історії унікальних покупців
CREATE TABLE homework_bigquery.daily_unique_users
(
  rep_date DATE,
  count_users INT64
);

--запит для додавання нових данних, не видаляючі старі
INSERT INTO homework_bigquery.daily_unique_users (rep_date, count_users)
```

```
SELECT CURRENT_DATE() AS rep_date,
COUNT(DISTINCT(user_id)) AS count_users
FROM `homework_bigquery.event_logs`
WHERE event_name='purchase'
AND DATE(event_time) BETWEEN CURRENT_DATE()-2 AND CURRENT_DATE();
```



Завдання 6



Оптимізований запит з використанням фільтрації за _PARTITIONDATE. Фільтрація за _PARTITIONDATE зменшить вартість запиту і підвищить швидкість.

```
--запит для додавання нових данних, не видаляючі старі
INSERT INTO homework_bigquery.daily_unique_users (rep_date, count_users)
SELECT CURRENT_DATE() AS rep_date,
COUNT(DISTINCT(user_id)) AS count_users
FROM `homework_bigquery.event_logs`
WHERE event_name='purchase'
AND _PARTITIONDATE BETWEEN CURRENT_DATE()-2 AND CURRENT_DATE();
```

This query will process 430 B when run.

Завдання 7

```
--створення CTE із масивом дій кожного користувача у межах дня
WITH users_events_per_day AS
(
SELECT user_id,
DATE(event_time) AS event_date,
ARRAY_AGG(event_name) AS events_per_day
FROM `homework_bigquery.event_logs`
GROUP BY user_id, event_date
)
--розгортаємо цей масив через UNNEST
SELECT user_id,
event_date,
event_name
FROM users_events_per_day
CROSS JOIN
UNNEST(events_per_day) AS event_name
```

Query results

Job information | Results | Visualization | JSON | Execution details

Row	user_id	event_date	event_name
1	user_001	2024-06-01	open_app
2	user_001	2024-06-01	search
3	user_001	2024-06-01	view_product
4	user_001	2024-06-01	add_to_cart
5	user_001	2024-06-01	purchase
6	user_002	2024-06-01	open_app
7	user_002	2024-06-01	view_product
8	user_002	2024-06-01	add_to_cart
9	user_002	2024-06-01	purchase